

Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

Learning objectives

- Create and run a simple Python file with basic input,output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

Files to create and commit in GitHub

stage01/
 smartcare_v01.py
 comparison.md
 reflection.md
 ai_usage.md

Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

Patient name, practitioner name and appointment time

What functions might be useful?

A function to book an appointment and a function to display recorded appointments

What could go wrong?

Duplicate bookings, blank or invalid information, incorrect appointment times, or missing practitioner details

What requirements are unclear?

The exact patient details to store, appointment status options, cancellation rules, practitioner availability rules and how duplicate bookings should be handled

Part B - Build a Human-Written Prototype: AI OFF

#task 1

Create and run a simple Python file with basic input,output statements

```
print("Welcome to SmartCare: Community Clinic Appointment Booking System!")
```

First Appointment

```
patient1_name = 'Alice Smith'
```

```
practitioner1_name = 'Dr. John Doe'
```

```
appointment1_time = '2024-07-20 10:00 AM'
```

```
print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time: {appointment1_time}")
```

Second Appointment

```
patient2_name = 'Bob Johnson'
```

```
practitioner2_name = 'Dr. Jane Roe'
```

```
appointment2_time = '2024-07-20 11:30 AM'
```

```
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time: {appointment2_time}")
```

#task1enhanced

Use lists, dictionaries and functions to enhance the Python file

```
appointments = []
```

```
def book_appointment(patient_name, practitioner_name, appointment_time):
```

```
    if not patient_name:
```

```
        raise ValueError("Patient name cannot be empty")
```

```
    appointment = {
```

```
        "patient": patient_name,
```

```
        "practitioner": practitioner_name,
```

```
        "time": appointment_time
```

```
    }
```

```
    appointments.append(appointment)
```

```
def display_appointments():
```

```
    if not appointments:
```

```
        print("No appointments recorded.")
```

```
        return
```

```
    for appointment in appointments:
```

```
        print(f"Patient: {appointment['patient']} | Practitioner:
```

```
{appointment['practitioner']} | Time: {appointment['time']}")
```

```
print("Welcome to SmartCare: The Clinical Appointment Booking System!")
```

```
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
```

```
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
```

```
display_appointments()
```

^ Now, run both programs , and identify at least five limitations.

1. The program does not prevent two appointments being booked for the same practitioner and time
2. It only check whether the patient name is blank; practitioner name and appointment time are not validated
3. Appointment information is only stored in memory while the program is running
4. There is no function to cancel or update an appointment
5. The program does not check practitioner availability before booking

Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

Act as a Python tutor.

I am learning introductory software technology.

Here is a small appointment-booking function.

1. Explain what the code does.
2. Identify three limitations.
3. Suggest improvements.
4. Do not rewrite the whole application.
5. Ask me two questions to test my understanding.

AI tutor notes: (From Microsoft CoPilot)

✓ 1. Explain what the code does

The function creates a **single appointment record** using a Python dictionary. Each appointment stores three pieces of information:

- **patient name**
- **practitioner name**
- **appointment time**

It then **appends** that dictionary to the global `appointments` list, which acts as your in-memory appointment book.

Your `display_appointments()` function loops through that list and prints each stored appointment.

This design is simple, readable, and perfect for beginners.

⚠ 2. Identify three limitations

These limitations come directly from your document and are accurate:

- **Duplicate bookings are allowed** — nothing stops the same practitioner being booked at the same time.
- **Weak validation** — only the patient name is checked; practitioner and time can be blank or invalid.
- **Temporary storage** — appointments disappear when the program ends because everything is stored in memory.

🔧 3. Suggest improvements

Here are small, controlled improvements that fit your prototype:

- Validate **all three fields** before storing an appointment
- Add a check to prevent **same practitioner + same time** duplicates
- Later (only if required), add simple update or cancellation functions

Your document already suggests this improvement clearly:

“Validate all required fields, check for duplicate practitioner/time bookings...”

🚫 4. Do not rewrite the whole application

No full rewrite — just small, focused enhancements that strengthen reliability without changing the structure.

❓ 5. Two questions to test your understanding

Here are two tutor-style questions that match your learning level:

1. **Why is a dictionary a good data structure for storing one appointment?**
2. **What logic could you add to stop a practitioner being booked at the same time twice?**

Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI. (From Microsoft CoPilot)

✓ Beginner-friendly appointment-booking function

Python ^ Copy

```
appointments = []

def add_appointment(patient_name, practitioner_name, appointment_time):
    # Basic validation to ensure all details are provided
    if not patient_name or not practitioner_name or not appointment_time:
        raise ValueError("All appointment details are required")

    # Create a dictionary to store one appointment
    appointment = {
        "patient": patient_name,
        "practitioner": practitioner_name,
        "time": appointment_time
    }

    # Store the appointment in the list
    appointments.append(appointment)

    return appointment
```

Part E - Compare Human and AI Versions

Question	Human version	AI version
Easy to understand?	Yes simple list, dictionary and functions.	Yes similar structure with one extra validation check
Runs successfully?	Yes	Yes with valid input
Uses only required features?	Yes	Yes no databased or GUI
Adds assumptions?	Minimal	Yes assumes all three fields must be provided
Handles errors?	Partly - blank patient name only	Partly checks all three required fields
Could I explain it?	Yes	Yes it uses concepts from the human version

Part F - Verify Behaviour

- Normal appointment

Accepted and stored successfully

- Blank patient name

Rejected with a value error in the human version and AI alternative

- Two appointments for the same practitioner/time

Both versions currently allow the duplicate booking, so this remains a limitation

- Strange input such as patient_name=None or appointment_time=None

patient_name=None is rejected. The human version can still accept appointment_time=None; the alternative rejects it.

Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient_name: raise ValueError("Patient name cannot be empty")

Chosen improvement: Validate all required appointment fields before storing the appointment.

Example: If not patient_name or not practitioner_name or not appointment_time: raise ValueError("All appointments details are required")

Reason: This is a small controlled change that prevents incomplete appointment records without adding features outside the current prototype.

Part H - Reflection (150-250 words)

What did you build before using AI?

What did AI help you understand?

Did AI make assumptions?

How did you verify the AI output?

What engineering work remained for you?

Before using AI, I built the SmartCare appointment prototype using basic Python variables, lists, dictionaries and functions. The program could store a patient name, practitioner name and appoint time, then display the recorded appointments. The AI tutor helped me understand how the booking function works and helped identify limitation such as duplicate bookings, weak input validation and temporary storage. Some AI suggestions could add assumptions, so I compared each suggestion with the SmartCare requirements instead of automatically accepting it. I verified the output by running the code and testing a normal appointment, a blank patient name but still allows some invalid or duplicate information. The engineering work that remained for me was deciding which improvements were actually required, testing the behaviour, checking the code against the case study and making sure I could explain the final code myself.

Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.

- Normal and unusual inputs tested.
- AI assistance documented.
- Reflection completed.
- I can explain my code.